大语言模型预训练

从原始文本到 Base Model 的完整过程

核心 一句话抓住本质

给模型看海量文本，在每个位置预测“下一个 Token”。预测错了就计算损失、反向传播并更新参数。这个循环重复数百万次，最终得到能够续写文本、掌握广泛语言模式与知识的 Base Model。



适用：本科生 / 研究生课堂讲授与课后复习

阅读路线：直觉 -> 数据 -> Token -> Transformer -> 概率 -> Loss -> Backprop -> Optimizer -> 大规模训练

1 先把预训练看成一个“自监督分类问题”

对 Decoder-only 大语言模型（如 GPT、Llama、Qwen 一类架构）而言，预训练最核心的任务是 Next Token Prediction：根据前面已经出现的 Token，预测下一个 Token。

$$P_\theta(x_t \mid x_{<t})$$

给定前缀 $x_{<t}$，模型输出真实下一个 Token x_t 的概率。

最小例子：一句话自动变成很多训练目标

原始文本：“浙江大学位于杭州。” 假设 Tokenizer 将它切成 4 个 Token：

已看到的前缀	应该预测的下一个 Token
<BOS>	浙江大学
<BOS> 浙江大学	位于
<BOS> 浙江大学 位于	杭州
<BOS> 浙江大学 位于 杭州	。

直觉 为什么叫“自监督”？

因为标签不需要人工额外标注。文本自身的后续 Token 就是监督信号：输入是前缀，标签是它真实出现的下一个 Token。

概率链式法则

$$P_\theta(x_{1:T}) = \prod_{t=1}^{T} P_\theta(x_t \mid x_{<t})$$

整段文本的概率可以拆成每个位置“下一个 Token 概率”的乘积。

因此，最大化真实训练文本的概率，等价于让每一个真实下一个 Token 的条件概率尽可能高。

2 数据：模型究竟在“读”什么？

预训练通常混合大量不同来源的文本。不同数据类型会给模型带来不同能力倾向。

数据类型	示例	可能学到的能力
网页	百科、机构网站、博客	常识、事实、自然语言表达
书籍	小说、教材、非虚构作品	长文本结构、叙事、逻辑
论文	学术论文、技术报告	专业知识、学术表达
代码	GitHub 等代码库	程序语法、API 用法、代码模式
问答/论坛	技术问答、社区讨论	对话表达、问题解决模式
数学文本	教材、题解、公式	符号模式、数学推理
多语言文本	中文、英文及其他语言	跨语言表示与翻译模式

数据进入训练前通常要做什么？

- HTML / Markdown / 文档解析：抽取正文，去掉导航、广告和模板噪声。
- 语言识别：按语言或领域做采样配比。
- 质量过滤：去掉乱码、低信息密度、机器拼接、垃圾页等。
- 去重：包括完全重复（exact dedup）和近似重复（near dedup）。
- 安全与隐私处理：过滤明显危险内容、个人敏感信息等。
- 数据混合与采样：不同来源按权重组成最终训练 corpus。

提醒 课堂提醒

“参数量大”不自动等于“模型强”。在同样参数规模下，数据质量、数据配比、去重和训练稳定性都可能显著影响最终效果。

为什么去重很重要？

如果互联网上同一句话出现 100 万次，它在总 Loss 中的权重会被人为放大，模型更容易发生过度记忆。近似重复内容也会造成类似问题。

完全重复

“Paris is the capital of France.”
同一文本被复制到大量网页。

近似重复

“浙江大学是一所著名高校。”
“浙江大学是一所非常著名的高校。”

3 Tokenizer：文字怎样变成模型能计算的数字？

神经网络不能直接处理“浙江大学”这样的字符串。首先要把文本切成 Token，再把每个 Token 映射为一个整数 ID。

text -> tokens -> token IDs

例如：“浙江大学位于杭州。” -> [浙江大学, 位于, 杭州, 。] -> [8123, 19382, 7261, 1773]

Token 不等于“一个字”，也不等于“一个单词”

原始字符串	可能的 Token 切分
unbelievable	un | believ | able
running	run | ning
人工智能	人工 | 智能
机器学习系统	机器 | 学习 | 系统

现代 LLM 常使用 BPE、SentencePiece 等 subword tokenizer。这样既能控制词表大小，又能组合出大量罕见词和新词。

把 Next Token Prediction 看成一次超大分类

如果词表大小 V = 100,000，那么模型在每一个位置都要回答：下一个 Token 在这 10 万个候选中究竟是哪一个？

$$z_t \in \mathbb{R}^V,\ V \approx 30{,}000 \sim 200{,}000$$

z_t 是第 t 个位置的 logits；词表越大，分类类别越多。

直觉 一个容易混淆的点

Token ID 只是离散编号。ID=8123 并不代表它“比 ID=100 大”。真正进入 Transformer 计算的是 Token 对应的连续向量 Embedding。

4 训练样本怎么构造：Input 和 Label 只错一位

假设 Token 序列是：[BOS, 我, 喜欢, 人工智能, 。]。训练时通常不会拆成 4 条独立样本，而是在一次 Forward 中同时预测所有位置。

序列	内容
Input	[BOS, 我, 喜欢, 人工智能]
Label	[我, 喜欢, 人工智能, 。]

Label = Input 向左移动 1 个 Token

同一个长度为 T 的序列，一次 Forward 可以提供约 T 个监督信号。

为什么这很高效?

假设 sequence length = 4096，则一个序列的一次 Forward 可以同时在数千个位置计算 next-token loss，而不是只产生一个标签。

例子 示例

输入 [BOS, 浙江大学, 位于, 杭州] 时，模型同时学习：BOS->浙江大学、浙江大学->位于、位于->杭州、杭州->句号。

长语料还需要切块与 Packing

- 长文档：按最大上下文长度切成多个 sequence。
- 短文本：可将多个文档 pack 到同一个 sequence 中，以减少 padding 浪费。
- 文档边界：通常用特殊 Token（如 EOS）区分，避免模型把两个独立文档误当成连续句子。

5 Embedding：把离散 Token ID 变成连续向量

设词表大小为 V，隐藏维度为 d，则 Token Embedding 矩阵可以写为：

$$E \in R^{(V \times d)}$$

例如 V = 100,000，d = 4096。

Token x_t 的向量表示就是查表：

$$e_t = E[x_t] \in R^d$$

例如 $x_t = 8123$，则取 Embedding 矩阵的第 8123 行。

如何给学生理解 Embedding?

可以把 Embedding 粗略理解为 Token 在高维连续空间中的“可学习坐标”。训练过程中，相关 Token 的向量及其后续上下文表示会逐渐形成结构。

注意 不要把 Embedding 讲得过度简单

“猫和狗的向量会很接近”只是方便建立直觉。真正的大模型语义主要存在于多层 Transformer 产生的上下文化表示中，而不只是静态 Embedding 表。

还需要位置信息

仅有 Token 向量，模型不知道“谁在前、谁在后”。因此模型还要编码位置信息。现代 LLM 常使用 RoPE（Rotary Position Embedding）等方法，把相对位置信息融入 Attention。

6 Transformer：让每个 Token 根据上下文重新表示自己

Embedding 进入 N 层 Transformer Block。每一层可以先用一个简化视角理解为两部分：Attention 负责“从上下文取信息”，MLP 负责“对信息做非线性变换”。

$$H^{(l+1)} = \text{TransformerBlock}_l(H^{(l)})$$

从第 0 层 Embedding 表示开始，逐层得到更丰富的上下文化表示。

Self-Attention 的核心公式

$$Q = H W_Q,\quad K = H W_K,\quad V = H W_V$$

$$\text{Attention}(Q,K,V) = \text{softmax}\left(\left(\frac{QK^T}{\sqrt{d_k}}\right) + M\right) V$$

M 是 causal mask；$\sqrt{d_k}$ 用于控制点积尺度。

Q / K / V 的课堂类比

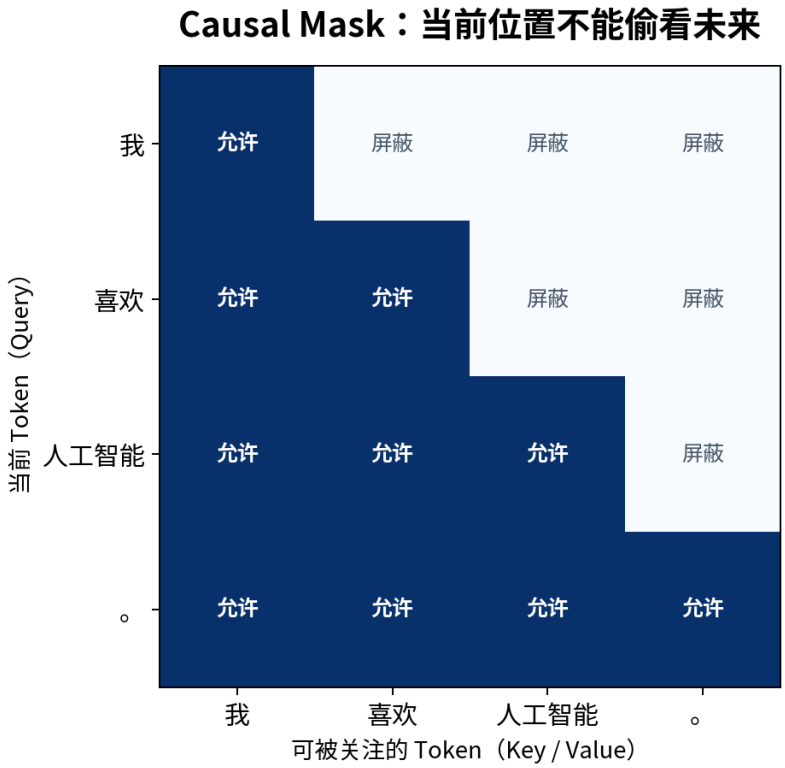
量	直觉问题	作用
Query (Q)	“我现在想找什么信息？”	当前 Token 发出查询
Key (K)	“我是什么类型的信息？”	决定自己与 Query 的匹配度
Value (V)	“如果关注我，实际拿走什么？”	被加权汇总的信息内容

直觉 一句话理解 Attention

每个 Token 都在上下文中做一次“软搜索”：用 Query 和所有可见位置的 Key 匹配，再按匹配权重加权读取 Value。

7 Causal Mask：为什么训练时模型不能“偷看答案”？

Decoder-only LLM 训练时必须保证第 t 个位置只能看到自己和前面的 Token，不能看到 t 之后的未来 Token。否则预测下一个 Token 就失去意义。



$$M_{ij} = 0 \ (j <= i), \quad M_{ij} = -\infty \ (j > i)$$

在 Softmax 前把未来位置加上 -∞，其注意力概率就会变成 0。

下三角可见性

当前 Token	我	喜欢	人工智能	。
我	允许	屏蔽	屏蔽	屏蔽
喜欢	允许	允许	屏蔽	屏蔽
人工智能	允许	允许	允许	屏蔽
。	允许	允许	允许	允许

关键 核心关系

正是 Causal Mask 把“整句话一次并行输入”与“每个位置只能使用历史信息”这两件事同时实现了。训练可以并行，但预测规则仍然是因果的。

8 LM Head + Softmax：从隐藏向量得到词表概率

Transformer 最后一层在每个位置输出隐藏向量 h_t ∈ R^d。它还不是“词的概率”，需要先通过 LM Head 映射到整个词表。

$$z_t = W_{LM} h_t + b, \quad z_t \in R^V$$

z_t 是 logits；若 V=100,000，则每个位置会得到 100,000 个分数。

Softmax 把 logits 变成概率

$$p_i = \exp(z_i) / \Sigma_j \exp(z_j)$$

所有候选 Token 的概率之和为 1。

数值例子 1：三个候选 Token

假设 logits = [2, 1, 0]，分别对应 A、B、C：

Token	Logit	Softmax 概率
A	2	约 0.665
B	1	约 0.245
C	0	约 0.090

提醒 重要区别

Logit 可以是任意实数，不是概率；Softmax 后才得到 0~1 之间、总和为 1 的概率分布。

9 Cross Entropy：模型“猜错了多少”？

对于真实下一个 Token y_t，最常用的单位置损失为负对数似然（等价于 one-hot 标签下的交叉熵）：

$$L_t = - \log P\theta(y_t \mid x_{<t})$$

正确 Token 的概率越高，Loss 越小。

概率和 Loss 的关系非常直观

正确 Token 的预测概率	Loss = -ln(p)	直觉
0.90	0.105	非常自信且正确，惩罚很小
0.50	0.693	不确定
0.10	2.303	正确答案概率很低
0.01	4.605	非常错误，惩罚很大

数值例子 2：“中国的首都是 ___”

候选 Token 只有 4 个。假设模型给出的概率如下：

候选 Token	预测概率
北京（正确）	0.738
上海	0.165
南京	0.061
东京	0.036

$$L = -\ln(0.738) \approx 0.304$$

如果模型只给“北京”1%概率，则 Loss 会变为约 4.605。

10 一整段文本的 Loss：对所有位置求和或求平均

对序列 x_1, x_2, …, x_T，每个位置都有一个 next-token loss。通常在有效 Token 上求平均：

$$L = -(1/T) \Sigma_{t=1}^{T} \log P\theta(x_t \mid x_{<t})$$

从最大似然角度再看一次

$$P\theta(x_{1:T}) = \prod_t P\theta(x_t \mid x_{<t})$$

$$\theta^* = \arg\max_\theta \Sigma_t \log P\theta(x_t \mid x_{<t})$$

$$\text{等价：} \theta^* = \arg\min_\theta -\Sigma_t \log P\theta(x_t \mid x_{<t})$$

也就是说，Next Token Prediction 本质上就是最大似然估计（MLE）的一个形式。

理解 研究生层面的关键理解

预训练目标并没有显式写“学习知识”“学习语法”“学习代码”“学习推理”。这些能力是模型为了持续降低 next-token loss，而在参数与内部表示中逐渐形成的。

11 完整数值演算：一句话如何走完一次训练 Step

训练文本：“人工智能正在改变世界。” 假设分词后：[BOS, 人工智能, 正在, 改变, 世界, 。]

步骤	具体内容
1. 构造 Input	[BOS, 人工智能, 正在, 改变, 世界]
2. 构造 Label	[人工智能, 正在, 改变, 世界, 。]
3. Embedding	每个 Token ID -> d 维向量
4. Transformer	每个位置根据之前的上下文得到 h_t
5. LM Head	h_t -> V 维 logits
6. Softmax	V 维 logits -> V 类概率
7. CE Loss	读取真实 Label 的概率并计算 -ln(p)
8. Backprop	对全部参数求梯度
9. Optimizer	AdamW 更新参数，进入下一个 batch

只看其中一个位置

上下文为 “[BOS, 人工智能, 正在]”，正确下一个 Token 是“改变”。假设模型给出：

候选	概率
改变（正确）	0.42
发展	0.18
成为	0.10
推动	0.07
其他 Token	0.23

$$L_3 = -\ln(0.42) \approx 0.868$$

同理计算其他位置的 loss，再对有效位置求平均，即得到这一条 sequence 对总 loss 的贡献。

12 反向传播：Loss 怎样“告诉参数往哪改”？

得到总 Loss 后，通过自动微分计算对所有可训练参数的梯度：

$$g = \nabla_\theta L = \partial L / \partial \theta$$

θ 包括 Attention、MLP、Embedding、LM Head 等处的大量权重。

Softmax + Cross Entropy 的一个漂亮结论

如果 p 是预测概率，y 是 one-hot 真实标签，则对 logits z 的梯度为：

$$\partial L / \partial z_i = p_i - y_i$$

数值例子 3：正确答案是“北京”

类别	预测 p	真实 y	梯度 p-y
北京	0.2	1	-0.8
上海	0.6	0	+0.6
南京	0.2	0	+0.2

直觉 直觉

梯度下降会倾向于把“北京”的 logit 推高，把“上海”“南京”的 logit 压低。这个误差信号再沿着 LM Head、Transformer、Embedding 一层层向前传播。

13 Optimizer：根据梯度真正修改参数

最简单的梯度下降是：

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta L$$

η 是 learning rate（学习率）。

现代大模型预训练常使用 AdamW。它会维护梯度的一阶矩与二阶矩，让不同参数的更新尺度更稳定。

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

$$\theta_{t+1} \approx \theta_t - \eta \cdot \hat{m}_t / (\mathrm{sqrt}(\hat{v}_t)+\varepsilon) + \text{weight decay}$$

课堂上理解“动量 + 自适应步长 + 权重衰减”即可。

Learning Rate 为什么很关键?

学习率过大	学习率过小
更新过猛，Loss 可能震荡、发散，甚至出现 NaN。	训练非常稳定，但需要更多 step 才能下降到同样的 Loss。

常见策略是先 Warmup，再逐渐 Decay，例如 cosine decay。Warmup 可以避免训练初期突然用过大的步长。

14 Batch、Sequence Length、Token 数：训练规模怎样算?

一个 Global Batch 有多少监督信号?

假设 global batch size = 1024 个 sequence，每个 sequence length = 4096：

$$1024 \times 4096 = 4{,}194{,}304 \approx 4.2\text{M tokens / step}$$

也就是说，一次参数更新可能汇总约 420 万个 Token 位置产生的梯度。

15T Tokens 大约对应多少 Step?

$$(15 \times 10^{12}) / (4 \times 10^{6}) \approx 3.75 \times 10^{6} \text{ steps}$$

这里只是数量级示例；真实训练取决于 global batch、重复 epoch、丢弃 token 等。

Gradient Accumulation

如果单次显存只能放 8 个 sequence，但想要等效 batch size = 128，可以连续累加 16 个 micro-batch 的梯度后再更新一次参数：

$$8 \times 16 = 128 \text{ effective batch size}$$

实践 训练日志里常见的量

step、tokens seen、train loss、validation loss、learning rate、gradient norm、throughput（tokens/s）、MFU 等。它们共同帮助判断训练是否稳定和高效。

15 为什么大模型预训练需要很多 GPU?

训练时显存不仅要存模型参数，还要存梯度、优化器状态以及前向过程的 activation。对几十亿到数百亿参数模型而言，这远远超过单张 GPU 的容量。

以 70B 参数为数量级直觉

$$70B \times 2\ Bytes \approx 140\ GB$$

这还只是 FP16/BF16 参数本身，不包含梯度、Adam 状态和激活。

并行方式	核心思想	典型目的
Data Parallel (DP)	不同 GPU 处理不同数据副本	扩大 global batch
Tensor Parallel (TP)	一个大矩阵拆到多个 GPU	单层参数放不下一张卡
Pipeline Parallel (PP)	不同层放到不同 GPU	按层切模型
Sequence Parallel	序列维度相关计算进一步拆分	降低激活/通信压力
Expert Parallel (EP)	MoE 的不同 Expert 分散到 GPU	支持大规模 MoE

工程 训练工程的本质

算法主线很简单，但把它稳定地扩展到数千张 GPU，需要解决通信、显存、容错、吞吐、数值稳定、检查点和数据供给等大量系统问题。

16 为什么“只猜下一个 Token”却能学到很多能力?

语法

模型反复看到 “I am / You are / He is / She is”。如果它持续预测出 “He are”，就会产生更高 Loss。为了降低 Loss，它必须逐渐捕获主谓一致等语言规律。

事实与知识

模型在不同上下文中大量看到 “France / Paris / capital / city” 的关联。为了正确预测后续 Token，会逐渐形成这些概念之间的统计与语义关系。

代码

看到大量代码后，模型为了预测函数体、关键字、括号和 API 调用，会逐渐学习程序语法和代码模式。

一定程度的推理模式

训练语料中包含数学题解、逻辑论证、程序执行轨迹等结构。模型为了降低 next-token loss，会学习可复用的模式和内部表示。但这不意味着“所有推理能力都由一个明确模块实现”。

结论 最重要的教学结论

预训练目标函数没有直接写“学知识”“学推理”。这些能力更像是模型为了把真实文本预测得更准而形成的副产品或涌现能力。

17 预训练结束后得到什么？—— Base Model

预训练完成后得到的是 Base Model。它最自然的行为是“续写文本”，而不一定可靠地把用户输入理解为一条必须服从的指令。

输入	Base Model 可能做什么
“请解释什么是牛顿第二定律。”	可能回答，也可能继续写“请再解释牛顿第一定律……”等文本模式
“把下面内容总结成三点：……”	不一定稳定遵守“三点”“简洁”等格式约束

为什么后面还要 SFT?

SFT 用高质量的 instruction-response 数据，让模型把“用户输入 -> 按要求回答”变成更强的行为模式。数学上，SFT 往往仍然使用 next-token cross entropy，只是数据分布和 loss mask 发生了变化。

维度	Pretraining	SFT
数据	海量自然文本	高质量 Prompt-Response
主要目标	学习通用语言与世界模式	学习指令遵循与对话行为
典型规模	Trillion 级 Tokens	百万级样本（数量级）
典型 Loss	Next-token CE	通常仍是 Next-token CE
产物	Base Model	Instruct Model

过渡 连接到下一节课

Pretraining 解决“会不会语言、有没有基础能力”；SFT 主要解决“会不会按人类指令回答”；RLHF / DPO 再进一步优化人类偏好、安全性和可用性。

18 公式速查表：把整节课压缩到一页

环节	公式 / 表达	含义
Token Embedding	e_t = E[x_t]	Token ID 查表得到 d 维向量
Q/K/V	Q=HW_Q, K=HW_K, V=HW_V	为 Self-Attention 准备三组投影
Attention	softmax((QK^T/√d_k)+M)V	匹配上下文并加权读取信息
LM Head	z_t = W_LM h_t + b	隐藏向量映射为 V 维 logits
Softmax	p_i=e^(z_i)/Σ_j e^(z_j)	logits 变成词表概率
单位置 CE	L_t=-log Pθ(y_t|x_<t)	惩罚正确 Token 概率过低
序列 Loss	L=-(1/T)Σ_t log Pθ(x_t|x_<t)	对所有有效位置聚合
梯度	g=∇_θL	Loss 对模型参数的导数
SGD 直觉	θ←θ-ηg	沿降低 Loss 的方向修改参数
PPL	PPL=e^L	语言模型常用困惑度指标

全流程最终版

Text -> Token -> Embedding -> Transformer -> Logits -> Softmax -> CE Loss -> Backprop -> AdamW -> New θ

总结 八个字记住预训练

读文本 -> 猜下词 -> 算误差 -> 改参数。工程上可以极其复杂，但算法主线就是这四步不断循环。

19 课堂讨论题与易错点

Q1：既然整句话一次输入，模型是不是已经看到了答案?

A：张量里确实包含整条序列，但 Causal Mask 保证位置 t 的 Attention 无法访问未来 Token，因此它在计算预测时仍然只能使用历史信息。

Q2：预训练是不是“无监督学习”？

A：更准确说是自监督学习。监督信号由数据自身自动构造，而不是人工标注。

Q3：SFT 和预训练是不是完全不同的算法?

A：不一定。很多 SFT 仍使用 next-token cross entropy，只是数据变成 Prompt-Response，并常常只对 Response 部分计算 loss。

Q4：模型是在“背诵互联网”吗?

A：既有泛化，也可能发生记忆。大量能力来自参数中的分布式统计表示，但高度重复或罕见文本也可能被记住。

Q5：为什么 Loss 降低不等于模型一定更安全?

A：预训练 Loss 主要衡量对数据分布的预测能力，并不直接等价于指令遵循、安全性、真实性或价值偏好。

Q6：为什么训练时可以并行、生成时却常常逐 Token 进行?

A：训练时真实目标序列已知，可对所有位置并行算 masked attention 和 loss；生成时下一个 Token 尚未知，必须先生成 x_t 才能继续生成 x_(t+1)。

20 一页总结：学生应该真正带走什么？

- 预训练数据不需要人工逐条标注，因为“后面的 Token”天然就是标签。
- Tokenizer 把文本变成 Token IDs；Embedding 把离散 ID 变成连续向量。
- Causal Transformer 只允许当前位置使用过去信息，通过 Attention 建模上下文。
- LM Head 把隐藏向量映射成整个词表的 logits，Softmax 得到概率。
- Cross Entropy 只关心：真实下一个 Token 的概率是否足够高。
- Backprop 计算梯度，AdamW 等优化器更新数十亿到数百亿参数。
- 这一过程在万亿级 Token 上重复数百万次，得到 Base Model。
- 语法、知识、代码和部分推理能力，是降低 next-token loss 过程中逐渐形成的。

$$\min_\theta \; L_{pretrain}(\theta) = -\frac{1}{N} \sum_n \sum_t \log P_\theta(x_t^{(n)} \mid x_{<t}^{(n)})$$

如果只记一个公式，就记这个。

收束 最后一句

Pretraining 让模型“会语言、懂模式”；SFT 让模型“会听指令”；RLHF / DPO 让模型“更符合人类偏好、更安全、更好用”。

—— 完 ——